

CS 505 Fall 2026 — Lab 1

32-bit Adder Design Space Exploration

Reproducible implementation and experimental report

October 01, 2026

1 Designs, experiment method, and verification

The experiment compares the supplied ripple-carry adder with two new standalone implementations, Kogge–Stone and Brent–Kung. Each has exactly the supplied `adder32(a,b,cin,sum,cout)` interface. Every design is evaluated both as a combinational adder and inside the supplied 32-bit synchronous counter, using the supplied ASAP7 LVT TT, Nangate45 typical, and Sky130HD TT cell libraries. This gives 18 configurations. The optional NAND-only Nangate library is excluded.

Both new designs first form $p_i = a_i \oplus b_i$ and $g_i = a_i b_i$. Prefix groups combine as $(G_H, P_H) \circ (G_L, P_L) = (G_H \vee (P_H \wedge G_L), P_H \wedge P_L)$. Kogge–Stone uses five prefix stages with distances 1, 2, 4, 8, and 16 (129 prefix-combine nodes). Brent–Kung uses five reduction stages followed by four distribution stages at distances 8, 4, 2, and 1 (57 combine nodes). Complete prefixes provide $c_0 = cin$, $c_{i+1} = G_{i:0} \vee (P_{i:0} \wedge cin)$, $sum_i = p_i \oplus c_i$, and $cout = c_{32}$. The original bit propagate values are retained for the sum. The RTL uses explicit combinational stages rather than an inferred generic addition operator. These node counts describe the source prefix graphs, not the number of cells after optimization and mapping.

Yosys maps each design into the selected library. OpenSTA first determines the minimum timing period, then estimates power with input activity 0.2 transitions per minimum period. Verilator simulates the mapped netlist using the supplied cell simulation models and produces the VCD used in a separate power analysis. Thus correctness, area, timing, and power refer to the same mapped implementation. All 18 mapped simulations passed; the analyzer also requires nonzero VCD activity annotation and complete reports.

The enhanced adder bench compares all 33 result bits, including carry-out, uses directed full/partial carry chains and overflow cases with both values of carry-in, and then 100,000 deterministic pseudorandom vectors. The counter bench checks reset results after clock edges, normal counting, and reset during counting. Icarus runs both benches against every RTL architecture. In addition, six Yosys SAT proofs pass: complete 33-bit addition for all 2^{65} combinations of inputs in each architecture, and the counter transition/reset property from an unconstrained register state for each architecture. The latter covers wraparound from `0xffffffff` without simulating billions of cycles.

The adder stimulus interval is 0.9855 ns (quantized to the testbench 1 ps precision); the counter clock period is 10 ns. These VCD workloads are fixed across architectures and technologies. Their activity rates differ from the probabilistic model, which uses each circuit’s minimum period. Consequently, differences between the two power columns reflect both activity assumptions and rate assumptions, not only circuit implementation. Specify timing checks are ignored by Verilator; this is functional gate-level activity, not a post-layout, delay-annotated measurement of silicon glitches.

1.1 Units and derived metrics

Timing reports use ps for ASAP7 and ns for Nangate45/Sky130HD; the analyzer reads `sta/units.log` and converts to seconds before calculations. OpenSTA’s `report_power` table is already in watts, even when `report_units` shows a different Liberty power unit. Area is the sum of mapped cell areas in the library, reported as μm^2 ; it does not include placement utilization or routed wire area. Let T be the minimum period in seconds, A mapped area, and P total power. Then throughput is $f = 1/T$, performance/area is f/A , and energy/operation is $E = PT$. Conventional throughput/watt is $f/P = 1/(TP)$ in operations/joule. The handout instead explicitly specifies T/P for “performance per Watt”; both quantities are retained with distinct names in the results and tables. Neither is silently substituted for the other.

The provided screenshot removes the official-template CSV instruction. The generated `results.csv` is therefore an auxiliary reproducibility artifact with our own explicit headers, not a purported official template.

2 Measured PPA results

The tables report total power (internal, switching, and leakage combined). All rows passed their mapped testbench. Energy here follows the handout’s PT definition using the minimum period. For VCD power, P is measured at the fixed simulation workload rate: its PT value is an assignment-defined metric, not the directly measured energy per simulated transaction.

2.1 Standalone adders

ASAP7 (7 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	11.36	0.8077	27.87	27.37	22.51	22.11
Kogge–Stone	19.39	0.3221	120.8	46.77	38.89	15.06
Brent–Kung	15.82	0.416	76.77	36.71	31.93	15.27

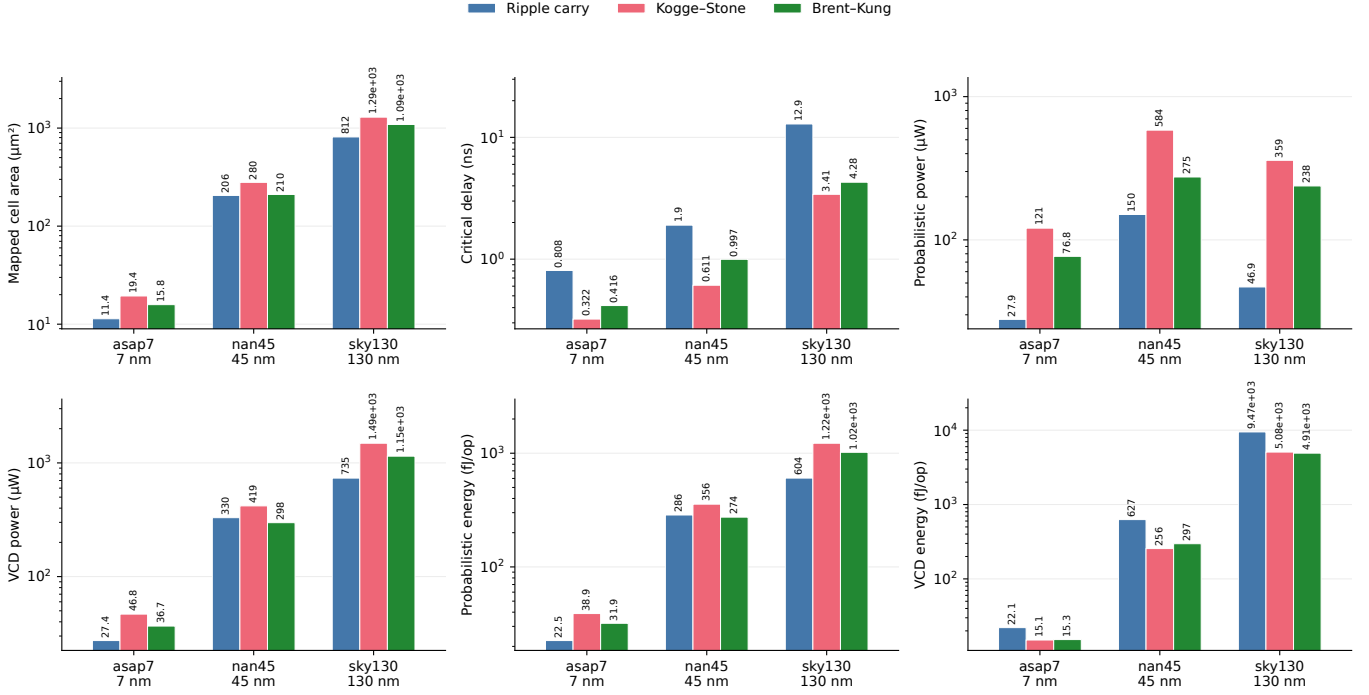
Nangate45 (45 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	205.6	1.902	150.4	329.8	286.1	627.3
Kogge–Stone	279.6	0.6107	583.6	418.7	356.4	255.7
Brent–Kung	210.4	0.9972	274.8	298	274	297.2

Sky130HD (130 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	812	12.88	46.92	735.2	604.3	9469
Kogge–Stone	1292	3.407	359.3	1490	1224	5075
Brent–Kung	1089	4.282	237.9	1146	1019	4909

32-bit standalone adders: mapped-cell comparisons



2.2 Counters

ASAP7 (7 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	18.56	0.3911	110.3	97.61	43.15	38.17
Kogge–Stone	18.62	0.3471	124.1	109.8	43.09	38.13
Brent–Kung	18.34	0.3679	117.1	103.7	43.08	38.15

Nangate45 (45 nm)

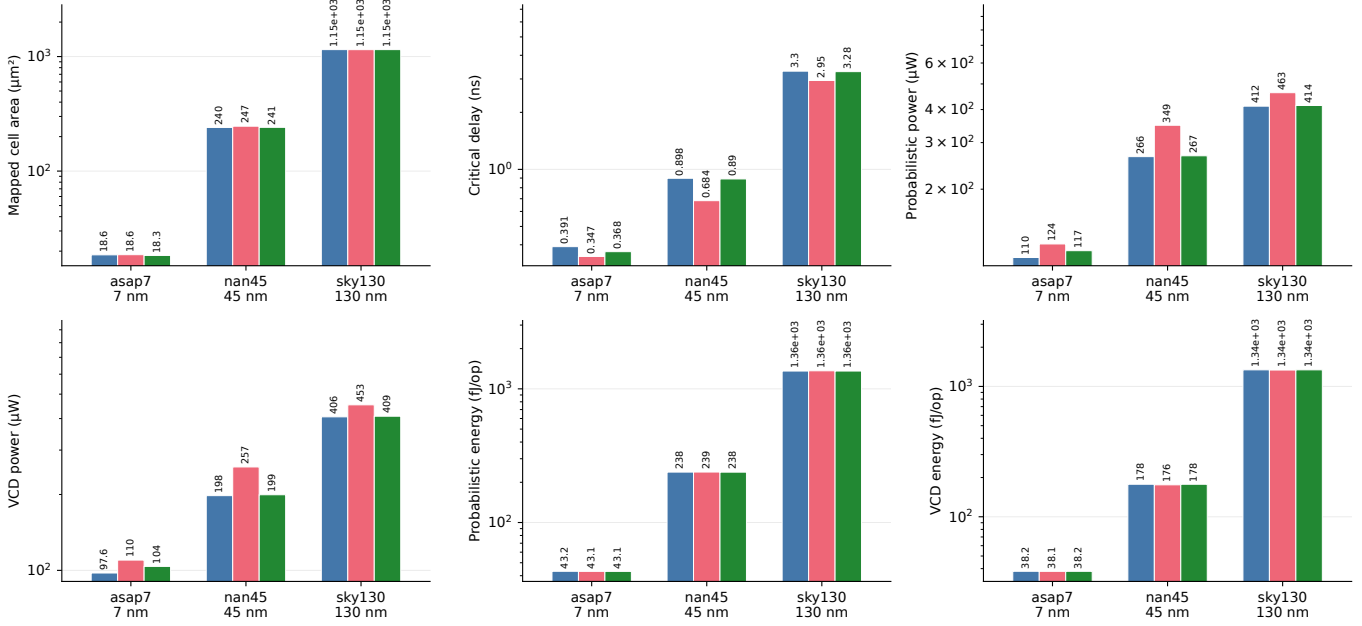
Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	240.5	0.8977	265.5	197.9	238.3	177.6
Kogge–Stone	246.6	0.6837	348.9	257.2	238.5	175.8
Brent–Kung	241	0.8902	267.5	199.5	238.1	177.6

Sky130HD (130 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	1150	3.298	412	406.4	1359	1340
Kogge–Stone	1149	2.946	463.3	453.5	1365	1336
Brent–Kung	1150	3.279	414.5	408.7	1359	1340

32-bit counters: mapped-cell comparisons

■ Ripple carry ■ Kogge-Stone ■ Brent-Kung



Logarithmic y axes. Energy = critical delay $\times$ power; excludes physical implementation and routing.

3 Qualitative questions

3.1 1. What is a netlist, and why analyze it?

`synth/netlist.v` describes the cells and connections that Yosys actually selected after flattening, optimization, and technology mapping. Behavioral Verilog and prefix equations do not identify cell sizes, input capacitances, timing arcs, leakage, or internal-power tables. Those depend on the selected library. For this experiment, the source node counts are fixed at 129 and 57, but the mapped area and delay vary by technology and counter constant folding. Analyzing the mapped netlist associates the circuit with those physical cell models and accounts for logic that was optimized away.

3.2 2. Two reasons an implementation might run faster than STA suggests

First, STA can include a structurally connected path that cannot be logically sensitized by a legal input transition (a false path), or use combinations of worst-case transitions that cannot occur together. A sensitization proof and appropriate constraints can remove that pessimism. A merely rare path is still relevant if correctness is required for every legal input. Second, real operating voltage, temperature, process, input slew, or load may be more favorable than the conditions used by the cell timing model. Cell characterization is a model; actual silicon under more favorable conditions can have a shorter delay. These are possible explanations, not measured speedups or proven false paths in these adders. Our supplied files use typical corners; no corner sweep or physical implementation was performed. Missing routed parasitics can also make this flow optimistic, so exceeding its frequency estimate is not guaranteed.

3.3 3. How can the original testbenches be improved?

Random testing alone misses very long carry chains with high probability, and the original adder bench checks only the low 32 sum bits. The implemented bench explicitly checks `{cout, sum}`, exercises every carry-chain length, tests both carry-in values, and makes failures terminate with a nonzero status. Deterministic pseudorandom inputs reproduce failures across simulators. The counter bench checks reset during operation as well as normal increments. Formal proof

covers every input/state, including carry-out and wraparound; mapped-netlist simulation then checks technology-model integration. Four-state Icarus simulation complements Verilator’s mostly two-state simulation.

3.4 4. Where does VCD activity come from, and what affects its power?

The VCD records signal changes while Verilator executes each synthesized netlist with the testbench stimuli. OpenSTA matches the correct `adder32_testbench/dut` or `counter32_testbench/dut` scope and uses annotated activity to estimate power. Random addition toggles many bits; counting has strongly unequal activity by bit position. Changing data patterns, input rate, reset fraction, simulated duration, dump coverage, or delay models changes the observed transition densities and hence switching/internal power. Leakage is less dependent on transition rate. A counter trace of roughly 100,000 increments mostly exercises low bits, while higher bits rarely toggle; formal correctness coverage does not turn that workload into a full activity sample of all states. The short directed adder prelude is included in its VCD and is identical across comparisons. The raw VCD files and annotation logs are retained, rather than assuming every pin received valid activity.

4 Quantitative questions

4.1 1. Three slowest unique start/end pairs

Appendix A lists the three slowest distinct pairs for every configuration. The original longest-path report is capped at 1,000 paths and can be dominated by repeated routes for a single pair. A supplementary `timing-unique-paths.rpt` enumerates the worst path per reachable pair by querying each primary-input or register-output source separately. Paths are ordered by reported slack; with the zero-period timing run this captures the limiting timing requirement. A pair can appear more than once because STA enumerates different intervening cell/pin routes and rise/fall transitions; the same endpoints need not identify the same timing path. For counters, setup/capture timing contributes to the minimum cycle time, so data-arrival time alone need not equal the required period. No repeated pair is counted twice in the appendix’s three-pair selection.

4.2 2. Performance versus performance/area and energy

For each technology, the following table gives Pearson correlations between raw throughput and the other metrics across our three standalone adders. For energy, negative correlation means higher throughput accompanies lower energy. With only three designs these values are descriptive, not evidence of statistical significance or a general law.

Technology	$r(f, f/A)$	$r(f, E_{prob})$	$r(f, E_{VCD})$
ASAP7 (7 nm)	0.973	0.998	-0.938
Nangate45 (45 nm)	0.962	0.837	-0.876
Sky130HD (130 nm)	0.983	0.998	-0.954

ASAP7 (7 nm): throughput order: Kogge–Stone, Brent–Kung, Ripple-carry; performance/area order: Kogge–Stone, Brent–Kung, Ripple-carry. Lowest probabilistic energy: Ripple-carry; lowest VCD energy: Kogge–Stone.

Nangate45 (45 nm): throughput order: Kogge–Stone, Brent–Kung, Ripple-carry; performance/area order: Kogge–Stone, Brent–Kung, Ripple-carry. Lowest probabilistic energy: Brent–Kung; lowest VCD energy: Kogge–Stone.

Sky130HD (130 nm): throughput order: Kogge–Stone, Brent–Kung, Ripple-carry; performance/area order: Kogge–Stone, Brent–Kung, Ripple-carry. Lowest probabilistic energy: Ripple-carry; lowest VCD energy: Brent–Kung.

A faster design can spend more area and draw more power, so the best raw delay need not maximize f/A or minimize PT . The probability model’s activity rate scales with $1/T$, while the VCD workload rate is fixed; energy rankings can consequently differ between power models.

4.3 3. Does architecture improvement scale equally at 7 and 130 nm?

Technology	Fastest	Slowest	Fast/slow speedup	KS/RCA	BK/RCA
ASAP7 (7 nm)	Kogge–Stone	Ripple-carry	2.508	2.508	1.942
Nangate45 (45 nm)	Kogge–Stone	Ripple-carry	3.115	3.115	1.907
Sky130HD (130 nm)	Kogge–Stone	Ripple-carry	3.781	3.781	3.008

The measured fastest/slowest speedup is 2.508 at 7 nm and 3.781 at 130 nm. The 7 nm speedup factor differs from the 130 nm factor by -33.67 percent, relative to the 130 nm factor. The architecture benefit does not scale equally between these nodes. These ratios, rather than absolute delay, show the relative architecture benefit. Technology libraries differ in relative XOR and AND/OR delays, complex-gate availability, drive strengths, fanout sensitivity, and ABC mapping choices. The same prefix equations can map to different gate chains. Faster processes therefore do not uniformly scale every architecture by the same factor. Our flow has no routed-wire extraction, so wire-layout explanations remain hypotheses rather than measured causes here.

4.4 4. Discrepancy between 7 nm counter power estimates

Architecture	P_{prob} (μW)	P_{VCD} (μW)	$ \Delta P $ (μW)	$ \Delta P /P_{prob}$ (%)
Ripple-carry	110.3	97.61	12.73	11.54
Kogge–Stone	124.1	109.8	14.3	11.52
Brent–Kung	117.1	103.7	13.39	11.43

By absolute difference, the smallest discrepancy is Ripple-carry and the largest is Kogge–Stone. By relative difference, the smallest is Brent–Kung and the largest is Ripple-carry.

Probabilistic input activity and propagated statistical assumptions differ from the correlated binary-count sequence. Counter high bits rarely toggle, the VCD covers only a finite counting interval, and reset changes the activity. The statistical power run uses each minimum period while the VCD clock is 10 ns. Clocked cell/internal power can therefore dominate the discrepancy. Library internal/leakage power models, and functional simulation’s limited glitch information, are additional model differences. Constant $b = 0$, $cin = 1$, and unused carry-out also allow synthesis to simplify an adder into an incrementer.

4.5 5. Counter area devoted to sequential elements

Sequential area is computed from mapped sequential-cell counts times each cell’s Liberty area, then divided by total mapped area. A cell is identified as sequential through its Liberty `ff/latch` definition, rather than its name alone.

Technology	Architecture	Seq. cells	A_{seq} (μm^2)	A_{total} (μm^2)	Seq. area (%)
ASAP7 (7 nm)	Ripple-carry	32	11.66	18.56	62.84
ASAP7 (7 nm)	Kogge–Stone	32	11.66	18.62	62.65
ASAP7 (7 nm)	Brent–Kung	32	11.66	18.34	63.59
Nangate45 (45 nm)	Ripple-carry	32	144.7	240.5	60.18
Nangate45 (45 nm)	Kogge–Stone	32	144.7	246.6	58.68
Nangate45 (45 nm)	Brent–Kung	32	144.7	241	60.04
Sky130HD (130 nm)	Ripple-carry	32	640.6	1150	55.71
Sky130HD (130 nm)	Kogge–Stone	32	640.6	1149	55.77
Sky130HD (130 nm)	Brent–Kung	32	640.6	1150	55.71

ASAP7 (7 nm): sequential share ranges from 62.65 to 63.59 percent (spread 0.95 percentage points).

Nangate45 (45 nm): sequential share ranges from 58.68 to 60.18 percent (spread 1.49 percentage points).

Sky130HD (130 nm): sequential share ranges from 55.71 to 55.77 percent (spread 0.06 percentage points).

Across all counters the share ranges from 55.71 to 63.59 percent. The largest within-node spread is 1.49 percentage points; the total cross-design/node spread is 7.88 percentage points. The ratio is therefore not constant across all designs and technologies.

The architectural state is always 32 bits, but the fraction is determined by flip-flop area relative to combinational increment/reset logic. Different adder structures and synthesis simplifications can change the denominator; different technologies change both flop and logic cell areas. Thus a constant state-bit count alone does not imply a constant area ratio. The measured table shows how closely that ratio agrees in each library.

5 Reproduction and artifacts

From the repository root run `python3 scripts/run_experiments.py -force`. It runs RTL simulation and formal verification, builds all 18 configurations, validates/parses the reports, generates plots and this PDF, and creates `submission.zip` containing the two new RTL files and `report.pdf`. The optional `-skip-build` regenerates analysis/report from existing builds; it still reruns RTL checks. A fresh build needs the supplied ilab tool and library paths, Icarus, Python with NumPy/Matplotlib, and pdfLaTeX. `build/build-manifest.json` records executable versions, exact commands, the original repository revision, and SHA-256 hashes of input source/library files. Per-target configuration files record build settings. The raw netlists, STA reports, simulation logs, VCDs, and proof logs remain under `build/`. `results/results.json`, `results/results.csv`, and figures are regenerable auxiliary files. No remote submission was performed.

Recorded tool versions:

yosys: Yosys 0.69+75 (git sha1 f0b945f63, Release, GNU /usr/bin/c++ 13.3.0)

opensta: 3.1.0

verilator: Verilator 5.053 devel rev v5.052-119-g014c9820d

iverilog: Icarus Verilog version 12.0 (stable) ()

A Three slowest unique endpoint pairs for every configuration

Names are copied from each `sta/timing-unique-paths.rpt`. The table reports the magnitude of worst slack normalized to ns (the timing requirement) and the number of occurrences of that endpoint pair in the original capped `timing-longest-paths.rpt`. Zero means that pair did not appear within the first 1,000 raw paths, although it is present in the complete unique-pair report.

A.1 adder32: Ripple-carry, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[31]	0.8077	1000
cin	sum[31]	0.8063	0
a[0]	sum[31]	0.8052	0

A.2 adder32: Kogge–Stone, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
a[0]	sum[30]	0.3221	48
b[0]	sum[30]	0.3207	48
cin	sum[30]	0.3196	48

A.3 adder32: Brent–Kung, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
a[0]	sum[31]	0.416	193
b[0]	sum[31]	0.4147	155
cin	sum[31]	0.4137	124

A.4 adder32: Ripple-carry, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[31]	1.902	955
a[0]	sum[31]	1.895	43
cin	sum[31]	1.891	2

A.5 adder32: Kogge–Stone, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[30]	0.6107	183
a[0]	sum[30]	0.6068	171
a[0]	sum[31]	0.5991	33

A.6 adder32: Brent–Kung, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[31]	0.9972	209
b[0]	sum[30]	0.9959	150
a[0]	sum[31]	0.9934	207

A.7 adder32: Ripple-carry, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
cin	cout	12.88	2
a[0]	cout	12.87	2
b[0]	cout	12.86	2

A.8 adder32: Kogge–Stone, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[30]	3.407	4
b[0]	cout	3.406	2
cin	sum[30]	3.403	4

A.9 adder32: Brent–Kung, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	cout	4.282	2
cin	cout	4.268	2
a[0]	cout	4.266	2

A.10 counter32: Ripple-carry, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.3911	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.3898	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.3883	10

A.11 counter32: Kogge–Stone, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q_29	0.3471	9
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q_31	0.3467	3
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_29	0.3463	9

A.12 counter32: Brent–Kung, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q_31	0.3679	16
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q_31	0.3667	16
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.3649	16

A.13 counter32: Ripple-carry, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.8977	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.8976	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.8918	8

A.14 counter32: Kogge–Stone, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.6837	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.681	8
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.6796	4

A.15 counter32: Brent–Kung, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.8902	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.8845	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q_31	0.8844	8

A.16 counter32: Ripple-carry, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	3.298	4
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q	3.273	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	3.272	4

A.17 counter32: Kogge–Stone, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	2.946	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_30	2.944	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_29	2.934	4

A.18 counter32: Brent–Kung, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q	3.279	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	3.278	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	3.277	4

B Both definitions of the requested efficiency metric

Here T/P is the handout’s literal quantity in s/W. Throughput/W is the conventional $1/(TP)$, shown in trillions of operations per joule (TOp/J). The complete CSV also contains throughput, performance/area, and both energies.

B.1 adder32

Technology	Architecture	T/P_{prob}	T/P_{VCD}	f/P_{prob}	f/P_{VCD}
ASAP7 (7 nm)	Ripple-carry	2.898e-05	2.951e-05	44.43	45.24
ASAP7 (7 nm)	Kogge–Stone	2.667e-06	6.887e-06	25.71	66.39
ASAP7 (7 nm)	Brent–Kung	5.419e-06	1.133e-05	31.32	65.49
Nangate45 (45 nm)	Ripple-carry	1.265e-05	5.769e-06	3.495	1.594
Nangate45 (45 nm)	Kogge–Stone	1.047e-06	1.459e-06	2.806	3.911
Nangate45 (45 nm)	Brent–Kung	3.629e-06	3.346e-06	3.649	3.365
Sky130HD (130 nm)	Ripple-carry	0.0002745	1.752e-05	1.655	0.1056
Sky130HD (130 nm)	Kogge–Stone	9.482e-06	2.287e-06	0.817	0.197
Sky130HD (130 nm)	Brent–Kung	1.8e-05	3.735e-06	0.9815	0.2037

B.2 counter32

Technology	Architecture	T/P_{prob}	T/P_{VCD}	f/P_{prob}	f/P_{VCD}
ASAP7 (7 nm)	Ripple-carry	3.544e-06	4.007e-06	23.17	26.2
ASAP7 (7 nm)	Kogge–Stone	2.796e-06	3.16e-06	23.21	26.23
ASAP7 (7 nm)	Brent–Kung	3.143e-06	3.549e-06	23.21	26.21
Nangate45 (45 nm)	Ripple-carry	3.381e-06	4.537e-06	4.196	5.63
Nangate45 (45 nm)	Kogge–Stone	1.959e-06	2.659e-06	4.192	5.688
Nangate45 (45 nm)	Brent–Kung	3.328e-06	4.463e-06	4.2	5.631
Sky130HD (130 nm)	Ripple-carry	8.005e-06	8.117e-06	0.7358	0.7461
Sky130HD (130 nm)	Kogge–Stone	6.359e-06	6.496e-06	0.7327	0.7486
Sky130HD (130 nm)	Brent–Kung	7.911e-06	8.023e-06	0.7358	0.7462